Prénom ..... Date .....

Trouver le plus grand élément

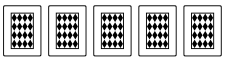
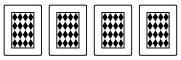
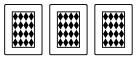
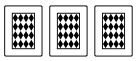
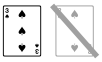
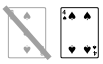
Problème : un tas de N cartes, faces cachées. On ne peut regarder et comparer que deux cartes à la fois.

Algorithme

1. Retourner la première carte du tas — elle devient le **maximum courant**
2. Tant que le tas n'est pas vide :
    1. Retourner la prochaine carte du tas
    2. La comparer avec le maximum courant
    3. Si elle est plus grande → elle devient le nouveau maximum courant, l'ancienne est écartée
    4. Sinon → la nouvelle carte est écartée
3. La carte restante en jeu est la plus grande du jeu

Trace

Exemple avec 5 cartes, tirées dans l'ordre : 3, 2, 4, K, 5

Tas	En jeu	
		Toutes les cartes sont faces cachées
		On retourne la 1ère carte → **maximum courant : 3**
		On retourne la 2ème carte
		3 > 2 → on garde 3, on écarte 2
		On retourne une nouvelle carte
		4 > 3 → on garde 4, on écarte 3 — **maximum courant : 4**
		On retourne une nouvelle carte
		K > 4 → on garde K, on écarte 4 — **maximum courant : K**
		On retourne la dernière carte
		K > 5 → on garde K, on écarte 5 — le tas est vide

Résultat : la plus grande carte est  — il a fallu **4 comparaisons** pour 5 cartes.

Complexité : pour N cartes, il faut exactement **N − 1 comparaisons**.

Trier un jeu de cartes

Problème : un tas de N cartes mélangées, faces cachées. On ne peut comparer que deux cartes à la fois. Trier le jeu du plus petit au plus grand.

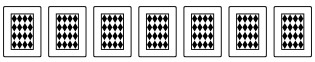
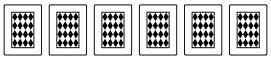
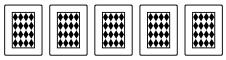
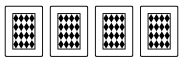
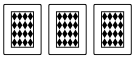
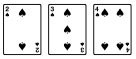
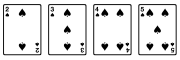
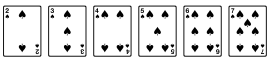
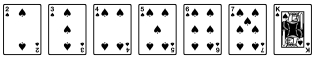
Algorithme

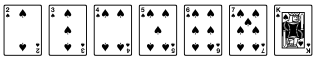
Idée : appliquer la recherche du minimum en boucle, en rangeant chaque minimum dans le résultat sans regarder.

1. Trouver le minimum du tas → le placer dans le résultat *(sans regarder l'ordre)*
2. Répéter avec le tas restant
3. Quand le tas est vide, le résultat est trié

Trace

Exemple : 7 cartes dans l'ordre initial 3, K, 4, 7, 2, 6, 5

Tas	Minimum		Résultat
		État initial	
		Minimum = 2 *(6 comparaisons)*	
		Minimum = 3 *(5 comparaisons)*	
		Minimum = 4 *(4 comparaisons)*	
		Minimum = 5 *(3 comparaisons)*	
		Minimum = 6 *(2 comparaisons)*	
		Minimum = 7 *(1 comparaison)*	
		Dernière carte : K	
		Le tas est vide — **le jeu est trié !**	

Résultat :  — il a fallu **21 comparaisons** pour trier 7 cartes.

Complexité : dans cet exemple : $6 + 5 + 4 + 3 + 2 + 1 = 21$ comparaisons pour 7 cartes.

En général, pour N cartes : $(N-1) + (N-2) + ... + 1 = \frac{N(N-1)}{2}$ comparaisons.

Fusionner deux jeux triés

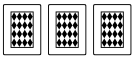
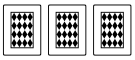
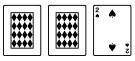
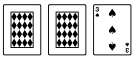
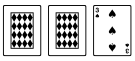
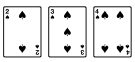
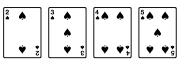
Problème : deux tas de cartes déjà triées, faces cachées. On ne peut comparer que deux cartes à la fois. Les fusionner en un seul tas trié.

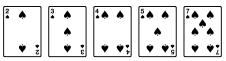
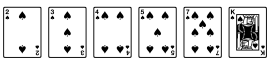
Algorithme

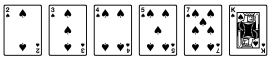
1. Retourner la carte du dessus de chaque tas
2. Prendre la plus petite → la placer dans le tas résultat
3. Retourner la prochaine carte du tas dont on vient de prendre
4. Répéter jusqu'à ce qu'un tas soit vide
5. Vider le tas restant dans le résultat

Trace

Exemple : fusionner [2, 4, K] et [3, 5, 7]

Tas			Résultat
T1		État initial : deux tas triés, faces cachées	
T2			
T1		On retourne la 1ère carte de chaque tas. On a 2 et 3	
T2			
T1		2 < 3 → on prend 2, on retourne la suivante de T1 On a 4 et 3	
T2			
T1		4 > 3 → on prend 3, on retourne la suivante de T2 On a 4 et 5	
T2			
T1		4 < 5 → on prend 4, on retourne la suivante de T1 On a K et 5	
T2			
T1		K > 5 → on prend 5, on retourne la suivante de T2 On a K et 7	
T2			

T1		K > 7 → on prend 7. On a K	
T2			
T1		T2 vide → on place les cartes restantes de T1 dans le résultat	
T2			

Résultat :  — il a fallu **5 comparaisons** pour fusionner 6 cartes.

Complexité : pour N cartes au total, il faut au plus **N − 1 comparaisons**.